

INSTITUTO TÉCNICO SALESIANO VILLADA
ESPECIALIDAD EN INFORMÁTICA



Programación Orientada a Objetos
PARADIGMA ORIENTADO A OBJETOS EN PYTHON

DOCENTES : Cortesini Pérez Luciano Tomás
Marchisone Mateo
CÁTEDRA : Programación I
CURSO : 4to año C

CÓRDOBA, ARGENTINA
14 de septiembre de 2026

Tabla de Contenidos

1. Introducción a la POO	3
1.1. De la Programación Estructurada a la POO	3
1.2. ¿Qué es un Objeto?	3
1.3. Clases y Objetos: Moldes e Instancias	3
2. Clases, Objetos y Atributos de Instancia	5
2.1. Definición de una Clase en Python	5
2.2. El Constructor <code>__init__</code> y el Parámetro <code>self</code>	5
2.3. Métodos de Instancia	6
2.3.1. Cálculos Derivados a partir de Atributos	6
2.3.2. Validación de Datos y Reglas de Negocio	6
2.3.3. Gestión del Estado Interno y Alternancia de Estados	7
2.4. Atributos con Estructuras de Datos Complejas	7
3. Atributos de Clase y Métodos Especiales	9
3.1. Atributos de Clase vs Atributos de Instancia	9
3.2. Métodos Especiales (Dunder Methods)	9
3.2.1. Representación como Cadena: <code>__str__</code> y <code>__repr__</code>	9
3.2.2. Sobrecarga de Operadores Aritméticos: <code>__add__</code>	10
3.2.3. Métodos Especiales <code>__len__</code> y <code>__call__</code>	10
4. Herencia y Polimorfismo	12
4.1. Herencia	12
4.2. Sobrescritura de Métodos y Uso de <code>super()</code>	12
4.3. Inspección de Tipos y Jerarquías	13
4.4. Polimorfismo	13
5. Encapsulamiento y Composición	16
5.1. Encapsulamiento y Ocultamiento de Información	16
5.1.1. Atributos Protegidos (<code>_atributo</code>)	16
5.1.2. Atributos Privados (<code>__atributo</code>) y Name Mangling	16
5.2. Propiedades (<code>@property</code> y Setters)	16
5.3. Relaciones entre Objetos: Composición y Agregación	17

Introducción a la POO

1.1. De la Programación Estructurada a la POO

En los primeros cursos de programación, el enfoque predominante es el paradigma estructurado. En la programación estructurada, las estructuras de datos (variables, listas, diccionarios) y los procedimientos (funciones) se mantienen separados. Las funciones reciben datos como argumentos, los procesan y retornan un resultado.

Sin embargo, a medida que los problemas de software aumentan en complejidad, la separación entre datos y funciones dificulta el mantenimiento y la escalabilidad del código. La *Programación Orientada a Objetos* (POO) propone una forma distinta de organizar y estructurar el código: agrupar los datos y las acciones que los manipulan dentro de una misma entidad llamada *objeto*.



Figura 1.1: Comparación entre Programación Estructurada y Programación Orientada a Objetos.

Como se ilustra en la Figura 1.1, la POO permite integrar el estado y el comportamiento en un bloque cohesivo, facilitando el modelado de sistemas complejos.

1.2. ¿Qué es un Objeto?

En el mundo real, un objeto es cualquier entidad física o abstracta que posee características y realiza acciones. Por ejemplo, un fibrón de pizarra tiene características como color, marca y masa, y realiza acciones como escribir o ser borrado. Del mismo modo, un automóvil tiene un modelo, patente, kilometraje y color, y acciones como acelerar, frenar o doblar.

En programación, un *objeto* es un modelo conceptual que almacena información y define comportamientos asociados. Todo objeto se compone de dos elementos fundamentales:

- *Atributos*: representan el estado o las características del objeto (por ejemplo, el saldo de una cuenta bancaria).
- *Métodos*: representan las acciones o procedimientos que el objeto puede ejecutar (por ejemplo, depositar o retirar dinero).

1.3. Clases y Objetos: Moldes e Instancias

Para crear objetos en un programa, primero debemos definir su estructura mediante una *clase*. Una clase funciona como un plano o plantilla arquitectónica que especifica qué atributos y métodos tendrán los objetos contruidos a partir de ella.

Un *objeto* es una *instancia* concreta de una clase. Aunque dos personas tengan un automóvil del mismo modelo (construidos con el mismo plano), cada automóvil es un objeto independiente con su propia patente, kilometraje acumulado y estado de desgaste.

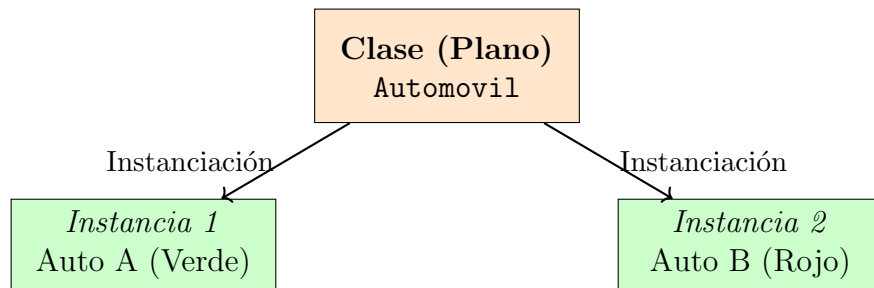


Figura 1.2: Relación entre una Clase (plantilla) y sus Instancias (objetos reales).

Como se observa en la Figura 1.2, una única definición de clase permite instanciar múltiples objetos con valores de atributos independientes en memoria.

Clases, Objetos y Atributos de Instancia

2.1. Definición de una Clase en Python

Para declarar una clase en Python se utiliza la palabra reservada `class`, seguida del nombre de la clase (escrito convencionalmente en formato *CamelCase*) y dos puntos. Si deseamos definir una clase vacía sin atributos ni métodos iniciales, empleamos la sentencia `pass` como marcador de posición sintáctico.

```
1 class Vehicle:
2     """Clase base que representa un vehiculo generico."""
3     pass
4
5 # Inspeccion del objeto clase
6 print(Vehicle)
```

Listing 2.1: Definición de una clase vacía y su instanciación.

En el Sección 2.1 se observa el uso de un *docstring* (cadena literal encerrada entre tres comillas), el cual permite documentar la función o propósito de la clase directamente en el código fuente.

2.2. El Constructor `__init__` y el Parámetro `self`

Cuando se crea una nueva instancia de una clase, Python ejecuta automáticamente un método especial de inicialización llamado `__init__`. Este método actúa como el constructor de la clase, reservando espacio y asignando los valores iniciales a los atributos del objeto.

El primer parámetro de todo método de instancia en Python debe ser obligatoriamente `self`. El parámetro `self` es una referencia explícita a la instancia concreta sobre la cual se está ejecutando el método.

```
1 class Vehicle:
2     """Representa un vehiculo con velocidad maxima y kilometraje.
3     """
4
5     def __init__(self, name, max_speed, mileage):
6         self.name = name
7         self.max_speed = max_speed
8         self.mileage = mileage
9
10 # Instanciacion de objetos independientes
11 v1 = Vehicle("Tesla Model S", 250, 18)
12 v2 = Vehicle("Toyota Corolla", 180, 45000)
13
14 print(f"Vehiculo 1: {v1.name}, Maxima: {v1.max_speed} km/h")
15 print(f"Vehiculo 2: {v2.name}, Maxima: {v2.max_speed} km/h")
```

Listing 2.2: Uso del constructor y asignación de atributos de instancia.

Al ejecutar la línea `v1 = Vehicle("Tesla Model S", 250, 18)`, Python invoca internamente a `Vehicle.__init__(v1, "Tesla Model S", 250, 18)`. La notación de punto (`v1.name`) permite acceder a los atributos específicos almacenados en esa instancia.

2.3. Métodos de Instancia

Los métodos de instancia son funciones definidas dentro de una clase que operan sobre los datos almacenados en los atributos de la instancia actual. A través del parámetro `self`, los métodos pueden leer o modificar el estado interno del objeto.

2.3.1. Cálculos Derivados a partir de Atributos

Un patrón habitual en POO es la creación de métodos que calculan nueva información utilizando los atributos almacenados en la instancia.

```

1 class Rectangle:
2     def __init__(self, length, width):
3         self.length = length
4         self.width = width
5
6     def area(self):
7         """Calcula el area del rectangulo."""
8         return self.length * self.width
9
10    def perimeter(self):
11        """Calcula el perimetro del rectangulo."""
12        return 2 * (self.length + self.width)
13
14 rect = Rectangle(10, 4)
15 print(f"Area: {rect.area()}")
16 print(f"Perimetro: {rect.perimeter()}")

```

Listing 2.3: Clase Rectangle con métodos de cálculo geométrico.

Al llamar a `rect.area()`, no es necesario pasar el argumento `self` explícitamente, ya que Python asocia automáticamente el objeto `rect` invocante.

2.3.2. Validación de Datos y Reglas de Negocio

Los métodos también se utilizan para imponer restricciones y validar las operaciones sobre el estado interno. Un ejemplo clásico es la gestión de cuentas bancarias con protección contra sobregiros.

```

1 class BankAccount:
2     def __init__(self, owner, balance=0.0):
3         self.owner = owner
4         self.balance = balance
5
6     def deposit(self, amount):
7         if amount > 0:
8             self.balance += amount
9             print(f"Deposito de {amount} exitoso. Saldo actual: {self.balance}")
10        else:
11            print("El monto a depositar debe ser positivo.")
12
13    def withdraw(self, amount):
14        if amount > self.balance:

```

```

15         print("Fondos insuficientes. Operacion cancelada.")
16     elif amount <= 0:
17         print("Monto de retiro invalido.")
18     else:
19         self.balance -= amount
20         print(f"Retiro de {amount} exitoso. Saldo restante: {
self.balance}")
21
22 cuenta = BankAccount("Juan", 1000)
23 cuenta.deposit(500)
24 cuenta.withdraw(2000) # Muestra mensaje de fondos insuficientes

```

Listing 2.4: Clase BankAccount con validación de saldo.

2.3.3. Gestión del Estado Interno y Alternancia de Estados

Los objetos pueden cambiar de estado mediante métodos dedicados a modificar variables booleanas o banderas internas.

```

1 class Light:
2     def __init__(self):
3         self.is_on = False
4
5     def turn_on(self):
6         self.is_on = True
7
8     def turn_off(self):
9         self.is_on = False
10
11    def toggle(self):
12        self.is_on = not self.is_on

```

Listing 2.5: Clase Light con métodos para alternar su estado.

2.4. Atributos con Estructuras de Datos Complejas

Los atributos de una instancia no se limitan a tipos primarios (enteros, cadenas, flotantes); también pueden almacenar colecciones completas como listas, diccionarios o conjuntos.

```

1 class Student:
2     def __init__(self, name, marks):
3         self.name = name
4         self.marks = marks # Lista de numeros flotantes o
enteros
5
6     def average(self):
7         if not self.marks:
8             return 0.0
9         return sum(self.marks) / len(self.marks)
10
11 s1 = Student("Alice", [85, 90, 78, 92, 88])

```

```
12 print(f"Promedio de {s1.name}: {s1.average():.2f}")
```

Listing 2.6: Clase Student almacenando una lista de notas.

Atributos de Clase y Métodos Especiales

3.1. Atributos de Clase vs Atributos de Instancia

En Python existen dos tipos de atributos según su ámbito de pertenencia:

- *Atributos de instancia*: pertenecen a un objeto específico y son inicializados habitualmente dentro del método `__init__` mediante `self.atributo`. Cada instancia mantiene su propia copia.
- *Atributos de clase*: se definen directamente en el cuerpo de la clase (fuera de cualquier método) y son compartidos por todas las instancias creadas a partir de dicha clase.

```

1 class Student:
2     # Atributo de clase (compartido por todos los estudiantes)
3     school_name = "Instituto Técnico Salesiano Villada"
4
5     def __init__(self, name, age):
6         # Atributos de instancia (unicos para cada objeto)
7         self.name = name
8         self.age = age
9
10 e1 = Student("Lucas", 16)
11 e2 = Student("Sofia", 16)
12
13 print(e1.school_name) # Muestra el valor de clase
14 print(e2.school_name)
15
16 # Modificación mediante el nombre de la clase
17 Student.school_name = "ITSV Cordoba"
18 print(e1.school_name) # Refleja el cambio en todas las
    instancias

```

Listing 3.1: Diferencia entre atributos de clase y de instancia.

Si se modifica un atributo de clase utilizando la referencia del nombre de la clase (`Student.school_name`), el cambio impacta en todas las instancias que no hayan sobrescrito dicho atributo a nivel de instancia.

3.2. Métodos Especiales (Dunder Methods)

Los métodos especiales, conocidos popularmente como *dunder methods* (por la abreviatura de *double underscore*), son métodos cuyo nombre inicia y finaliza con dos guiones bajos, como por ejemplo `__init__`. Python los utiliza para sobrecargar operadores y personalizar comportamientos integrados del lenguaje.

3.2.1. Representación como Cadena: `__str__` y `__repr__`

Cuando se imprime un objeto mediante `print()` o se convierte a cadena con `str()`, Python busca el método `__str__`. Cuando se inspecciona en la consola interactiva o dentro de colecciones, se utiliza `__repr__`.

- `__str__`: pensado para presentar una salida amigable y legible para el usuario final.
- `__repr__`: pensado para proporcionar una representación inequívoca, útil para desarrolladores durante el proceso de depuración.

```

1 class Book:
2     def __init__(self, title, author):
3         self.title = title
4         self.author = author
5
6     def __str__(self):
7         return f"'{self.title}' escrito por {self.author}"
8
9     def __repr__(self):
10        return f"Book(title='{self.title}', author='{self.author}')"
11
12 libro = Book("Python Crash Course", "Eric Matthes")
13 print(str(libro))    # Invoca __str__
14 print(repr(libro))  # Invoca __repr__

```

Listing 3.2: Implementación de `__str__` y `__repr__`.

3.2.2. Sobrecarga de Operadores Aritméticos: `__add__`

La sobrecarga de operadores permite definir cómo se comportarán los operadores matemáticos estándar (como +, -, *) cuando se apliquen sobre objetos de nuestras propias clases.

```

1 class Vector2D:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6     def __add__(self, other):
7         if isinstance(other, Vector2D):
8             return Vector2D(self.x + other.x, self.y + other.y)
9         return NotImplemented
10
11     def __str__(self):
12         return f"Vector({self.x}, {self.y})"
13
14 v1 = Vector2D(2, 4)
15 v2 = Vector2D(3, 1)
16 v3 = v1 + v2    # Invoca v1.__add__(v2)
17 print(v3)      # Salida: Vector(5, 5)

```

Listing 3.3: Sobrecarga del operador suma mediante `__add__`.

3.2.3. Métodos Especiales `__len__` y `__call__`

El método `__len__` permite a una clase responder a la función integrada `len()`.

```
1 class ShoppingCart:
2     def __init__(self):
3         self.items = []
4
5     def add_item(self, item):
6         self.items.append(item)
7
8     def __len__(self):
9         return len(self.items)
10
11 cart = ShoppingCart()
12 cart.add_item("Manzana")
13 cart.add_item("Pan")
14 print(f"Cantidad de productos: {len(cart)}")
```

Listing 3.4: Uso de `__len__` para calcular la cantidad de elementos.

Por otro lado, el método `__call__` permite que una instancia de una clase sea invocada como si fuera una función.

```
1 class Multiplier:
2     def __init__(self, factor):
3         self.factor = factor
4
5     def __call__(self, x):
6         return x * self.factor
7
8 duplicar = Multiplier(2)
9 print(duplicar(5)) # Retorna 10
```

Listing 3.5: Instancia invocable con `__call__`.

Herencia y Polimorfismo

4.1. Herencia

La *herencia* es un mecanismo fundamental de la POO que permite crear nuevas clases basadas en clases existentes. La clase original se denomina *superclase* o *clase padre*, mientras que la nueva clase se llama *subclase* o *clase hija*.

La subclase hereda automáticamente todos los atributos y métodos de la superclase, lo que fomenta la reutilización de código y permite establecer jerarquías conceptuales del tipo “*es-un*” (por ejemplo, un colectivo *es un* vehículo).

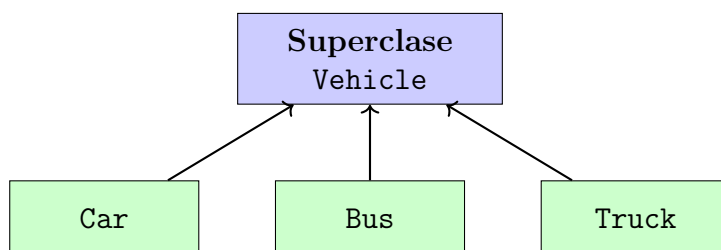


Figura 4.1: Jerarquía de herencia a partir de la superclase Vehicle.

```
1 class Vehicle:
2     def __init__(self, name, max_speed):
3         self.name = name
4         self.max_speed = max_speed
5
6     def display_info(self):
7         print(f"Vehiculo: {self.name}, Velocidad Maxima: {self.
8             max_speed} km/h")
9
10 # Bus hereda de Vehicle
11 class Bus(Vehicle):
12     pass
13
14 b = Bus("Mercedes Benz", 120)
15 b.display_info() # Metodo heredado de Vehicle
```

Listing 4.1: Sintaxis básica de herencia en Python.

4.2. Sobrescritura de Métodos y Uso de super()

En muchas situaciones, una subclase necesita adaptar o extender el comportamiento de un método heredado. Este proceso se denomina *sobrescritura de métodos* (*method overriding*).

Para reutilizar la lógica de la superclase en lugar de reescribirla por completo, Python proporciona la función `super()`.

```
1 class Vehicle:
2     def __init__(self, name, max_speed, mileage):
3         self.name = name
```

```

4         self.max_speed = max_speed
5         self.mileage = mileage
6
7     def fare(self):
8         return self.mileage * 10
9
10    class Bus(Vehicle):
11        def __init__(self, name, max_speed, mileage, capacity=50):
12            # Llama al constructor de la superclase Vehicle
13            super().__init__(name, max_speed, mileage)
14            self.capacity = capacity
15
16        def fare(self):
17            # Sobrescribe el metodo agregando un 10% de mantenimiento
18            base_fare = super().fare()
19            total_fare = base_fare + (base_fare * 0.10)
20            return total_fare
21
22    bus = Bus("Escolar", 90, 100, 40)
23    print(f"Tarifa del colectivo: ${bus.fare()}")

```

Listing 4.2: Uso de `super()` para extender el constructor y métodos.

4.3. Inspección de Tipos y Jerarquías

Python dispone de funciones integradas para consultar la pertenencia de un objeto a una clase o jerarquía:

- `type(obj)`: retorna la clase exacta a la cual pertenece un objeto.
- `isinstance(obj, Clase)`: comprueba si un objeto es una instancia de la clase especificada o de una subclase derivada de ella.
- `issubclass(Subclase, Superclase)`: verifica si una clase hereda de otra superclase.

```

1    b = Bus("Urbano", 80, 50)
2
3    print(type(b))                # <class '__main__.Bus'>
4    print(isinstance(b, Bus))      # True
5    print(isinstance(b, Vehicle))  # True (por herencia)
6    print(issubclass(Bus, Vehicle)) # True

```

Listing 4.3: Uso de `isinstance()` e `issubclass()`.

4.4. Polimorfismo

El *polimorfismo* es la capacidad que poseen distintos objetos de responder al mismo mensaje o llamada de método, ejecutando cada uno su propia implementación específica.

En Python, el polimorfismo se aplica frecuentemente mediante el principio de *Duck Typing* ("Si camina como un pato y suena como un pato, entonces es un pato"). Esto significa que a una función no le importa la clase exacta del objeto recibido, sino que este responda a los métodos invocados.

```

1 class Dog:
2     def speak(self):
3         return "Guau!"
4
5 class Cat:
6     def speak(self):
7         return "Miau!"
8
9 def hacer_sonar(animal):
10     # Invoca speak() de forma polimorfica sin importar el tipo
    exacto
11     print(animal.speak())
12
13 mascotas = [Dog(), Cat()]
14 for m in mascotas:
15     hacer_sonar(m)

```

Listing 4.4: Polimorfismo con jerarquía de animales.

Un ejemplo aplicado al cálculo de salarios en un sistema de recursos humanos ilustra el valor del polimorfismo:

```

1 class Employee:
2     def __init__(self, name):
3         self.name = name
4
5     def calculate_salary(self):
6         raise NotImplementedError("Las subclases deben
    implementar este metodo.")
7
8 class FullTimeEmployee(Employee):
9     def __init__(self, name, monthly_salary):
10         super().__init__(name)
11         self.monthly_salary = monthly_salary
12
13     def calculate_salary(self):
14         return self.monthly_salary
15
16 class PartTimeEmployee(Employee):
17     def __init__(self, name, hours_worked, hourly_rate):
18         super().__init__(name)
19         self.hours_worked = hours_worked
20         self.hourly_rate = hourly_rate
21
22     def calculate_salary(self):
23         return self.hours_worked * self.hourly_rate
24
25 empleados = [
26     FullTimeEmployee("Carlos", 350000),
27     PartTimeEmployee("Ana", 80, 2500)
28 ]
29
30 for e in empleados:

```

```
31 print(f"Empleado: {e.name}, Salario: ${e.calculate_salary()}"  
    )
```

Listing 4.5: Polimorfismo aplicado al cálculo de salarios.

Encapsulamiento y Composición

5.1. Encapsulamiento y Ocultamiento de Información

El *encapsulamiento* es el principio de la POO que consiste en ocultar los detalles internos de implementación de un objeto y restringir el acceso directo a sus atributos desde el exterior del código de la clase. Esto evita que los datos internos sean modificados de forma arbitraria o inconsistente.

A diferencia de lenguajes como C++ o Java, Python no posee palabras clave como `private` o `protected` a nivel de compilador, sino que emplea convenciones de nombres para indicar la privacidad de los miembros.

5.1.1. Atributos Protegidos (`_atributo`)

Un atributo cuyo nombre comienza con un único guion bajo (por ejemplo, `_balance`) indica convencionalmente que es un miembro protegido. Aunque Python permite acceder a él desde el exterior, la regla indica que solo debería ser utilizado internamente por la clase y sus subclases.

5.1.2. Atributos Privados (`__atributo`) y Name Mangling

Cuando un atributo comienza con dos guiones bajos (por ejemplo, `__balance`), Python activa un mecanismo llamado *Name Mangling* (modificación de nombres). El interprete transforma internamente el nombre del atributo agregando el prefijo `_NombreDeClase`, lo que dificulta su acceso o modificación accidental desde afuera.

```

1 class BankAccount:
2     def __init__(self, owner, balance):
3         self.owner = owner
4         self._protected_data = "Dato protegido"
5         self.__balance = balance # Atributo privado
6
7 cuenta = BankAccount("Laura", 5000)
8
9 print(cuenta.owner)           # Acceso directo permitido
10 print(cuenta._protected_data) # Permitido por sintaxis, pero
    desaconsejado
11
12 # El acceso a cuenta.__balance arroja AttributeError
13 # Para acceder forzosamente (mangling): print(cuenta.
    _BankAccount__balance)

```

Listing 5.1: Diferencia entre atributos protegidos y privados.

5.2. Propiedades (`@property` y `Setters`)

En lugar de escribir métodos de acceso tradicionales al estilo Java (como `get_balance()` y `set_balance()`), Python ofrece una alternativa elegante y concisa mediante decoradores de *propiedades*: `@property` para el getter y `@propiedad.setter` para el setter.

Las propiedades permiten acceder y modificar atributos privados mediante la notación habitual de atributos (`obj.balance = 100`), pero ejecutando código de validación transparente detrás de escena.

```

1 class BankAccount:
2     def __init__(self, balance):
3         self._balance = balance
4
5     @property
6     def balance(self):
7         """Getter: retorna el saldo protegido."""
8         return self._balance
9
10    @balance.setter
11    def balance(self, amount):
12        """Setter: valida que el saldo no sea negativo."""
13        if amount < 0:
14            raise ValueError("El saldo no puede ser negativo.")
15        self._balance = amount
16
17 cuenta = BankAccount(1000)
18 print(f"Saldo inicial: {cuenta.balance}") # Invoca el getter
19
20 cuenta.balance = 1500 # Invoca el setter con
    validacion
21 print(f"Nuevo saldo: {cuenta.balance}")

```

Listing 5.2: Uso de `@property` para validar la asignación de saldo.

5.3. Relaciones entre Objetos: Composición y Agregación

Mientras que la herencia modela una relación del tipo “*es-un*” (un colectivo *es un* vehículo), la *composición* y la *agregación* modelan relaciones del tipo “*tiene-un*”.

La composición ocurre cuando una clase compleja contiene instancias de otras clases como atributos propios, coordinando la interacción entre dichos objetos.



Figura 5.1: Relación de Composición: Un Vuelo contiene una lista de Pasajeros.

A continuación se presenta una clase `Flight` que gestiona la reserva y capacidad ocupada por objetos `Passenger`:

```

1 class Passenger:
2     def __init__(self, name, passport_number):
3         self.name = name
4         self.passport_number = passport_number
5
6 class Flight:

```

```

7     def __init__(self, flight_number, capacity):
8         self.flight_number = flight_number
9         self.capacity = capacity
10        self.passengers = [] # Lista que almacenara objetos
11        Passenger
12
13    def add_passenger(self, passenger):
14        if len(self.passengers) < self.capacity:
15            self.passengers.append(passenger)
16            print(f"Pasajero {passenger.name} abordado con exito.
17        ")
18            return True
19        else:
20            print("Vuelo completo. No se pudo agregar al pasajero
21        .")
22            return False
23
24    p1 = Passenger("Carlos Gomez", "AB123456")
25    p2 = Passenger("Maria Lopez", "CD789012")
26
27    vuelo = Flight("AR1302", 2)
28    vuelo.add_passenger(p1)
29    vuelo.add_passenger(p2)

```

Listing 5.3: Ejemplo de Composición con clases Flight y Passenger.

Del mismo modo, la composición permite implementar patrones donde una clase administradora procesa de forma polimórfica una colección de objetos contenidos en su interior:

```

1 class Animal:
2     def __init__(self, name):
3         self.name = name
4
5     def eat(self):
6         return f"{self.name} esta comiendo."
7
8 class Zoo:
9     def __init__(self):
10        self.animals = []
11
12    def add_animal(self, animal):
13        self.animals.append(animal)
14
15    def feed_all(self):
16        for animal in self.animals:
17            print(animal.eat())
18
19    zoo = Zoo()
20    zoo.add_animal(Animal("Leon"))
21    zoo.add_animal(Animal("Panda"))
22    zoo.feed_all()

```

Listing 5.4: Clase Zoo administrando una colección polimórfica de animales.

Bibliografía

- [Phi18] Dusty Phillips. *Python 3 Object-Oriented Programming*. Packt Publishing Ltd, 2018.
- [Mat19] Eric Matthes. *Python Crash Course: A Hands-On, Project-Based Introduction to Programming*. No Starch Press, 2019.